

Privacy-Preserving Recommendation Systems: A Survey of Cryptographic Approaches

Literature Survey, CS670

Mainak Sarkar Shrasti Dwivedi Anushka Gupta Aditya Anand

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
31 August 2026

Abstract

[**TODO:** Write this last. Roughly 200 words, four moves: (1) recommender systems leak consumption history, and the data is both valuable and dangerous; (2) two literatures have grown up separately — private *training* of the model, and private *retrieval* of the recommended item; (3) PIRSONA is close to the only system spanning both and now predates the whole modern private-retrieval line, while NUDGE is the state of the art on training and explicitly delegates the fetch to “other means”; (4) this survey maps both literatures, identifies the gap between them, and states what we propose to build. Do not oversell. Name the two base papers explicitly — the reader is the author of one of them.]

Contents

1	Introduction	3
1.1	Two halves of one problem	3
1.2	The apparent contradiction	3
1.3	Scope of this survey	3
1.4	Contributions of this survey	3
1.5	Roadmap	3
2	Background	3
2.1	Collaborative filtering and matrix factorisation	4
2.2	The recommendation pipeline	4
2.3	What the adversary sees	4
2.4	Utility, and why it constrains the cryptography	4
3	A Taxonomy of Approaches	4
3.1	Axis 1: the trust model	5
3.2	Axis 2: the privacy notion	5
3.3	Axis 3: which pipeline stage is protected	5
3.4	Axis 4: the adversary	5
3.5	Reading the table	5

4	Cryptographic Building Blocks	6
4.1	Secret sharing	6
4.2	Function secret sharing and distributed point functions	6
4.3	FSS as a comparison gate	6
4.4	Private information retrieval	6
4.5	Oblivious RAM	7
4.6	Fixed-point arithmetic over rings	7
5	Private Training	7
5.1	Matrix factorisation, in the clear	7
5.2	Garbled circuits: Nikolaenko et al.	7
5.3	PIRSONA: four-party Boolean factorisation	7
5.4	NUDGE: power iteration as a matrix-vector program	7
5.4.1	The two protocols that cost anything	8
5.5	The non-cryptographic alternatives	8
5.6	Where training now stands	8
6	Private Retrieval	8
6.1	The problem: private nearest-neighbour search	8
6.2	Linear-scan approaches	9
6.3	Sublinear and indexed approaches	9
6.4	Relaxing the guarantee	9
6.5	Private top- k as a service	9
6.6	An observation about the recommendation setting	9
7	Systems That Attempt Both Halves	10
7.1	PIRSONA as an end-to-end system	10
7.2	Partial attempts	10
7.3	The intersection is nearly empty	10
8	How This Literature Is Evaluated	11
8.1	Metrics that matter	11
8.2	Network models	11
8.3	Baselines	11
8.4	The standard we hold ourselves to	11
9	Open Problems	12
9.1	Malicious security at scale	12
9.2	Reducing the non-collusion assumption	12
9.3	The delivery stage	12
9.4	Leakage through the recommendation itself	12
9.5	Evaluation and comparability	12
9.6	Scope of our own follow-on work	12
10	Conclusion	13

1 Introduction

[**TODO:** Open with the concrete harm, not with a definition. *PIRSONA* §1 does this well — consumption data is invaluable to “unscrupulous marketers, identity thieves... and hostile foreign agents”. Borrow the framing, not the words.]

1.1 Two halves of one problem

[**TODO:** The central framing device of the whole survey. A recommender that protects its users must do two separate things: (i) *TRAIN* a model on private ratings without learning them, and (ii) *DELIVER* the recommended item without learning which item. Doing only (i) is not enough — the fetch betrays the recommendation. Doing only (ii) is not enough — the model was built from exposed data. Establish this here and refer back to it in every later section.]

1.2 The apparent contradiction

[**TODO:** Why this looked impossible: *PIR* hides WHICH item a user fetches, while collaborative filtering works precisely by relating users to each other through what they fetched. *PIRSONA* §1 states this tension explicitly — quote it, then say it has been resolved twice over, in two different ways, by two literatures that did not talk to each other.]

1.3 Scope of this survey

[**TODO:** State what is in and what is out. IN: MPC- and *PIR*-based approaches; matrix factorisation; private nearest-neighbour retrieval; the leakage that survives. OUT: pure differential privacy without a cryptographic layer; trusted hardware (mentioned in §2 for contrast only); neural recommenders. Say why the cuts were made — an unjustified scope cut reads as an unread paper.]

1.4 Contributions of this survey

[**TODO:** Be honest: a survey’s contribution is a map and a gap, not a result. Claim exactly three things: (1) the first side-by-side treatment of the private-training and private-retrieval literatures for recommendation; (2) identification of a concrete, author-acknowledged gap between them; (3) two cheap, checkable research questions arising from it (N1, N2 — see ??).]

1.5 Roadmap

[**TODO:** One short paragraph mapping §2–§9. Make the arc audible: primitives, then training, then retrieval, then who tried both, then the gap.]

2 Background

[**TODO:** Half a page of framing: recommendation is a data-mining problem whose input is, by construction, a record of what people consume. State that the survey organises the field by *PIPELINE STAGE* and that this section establishes that framing.]

[**TODO:** FIGURE (TikZ): the four-stage pipeline as a horizontal flow, with what an adversary observes annotated beneath each stage. This figure carries the survey’s organising idea and is referenced from sections 3, 5, 6 and 7. Keep it schematic — boxes, arrows, and leakage labels.]

Figure 1: The recommendation pipeline and what is exposed at each stage.

2.1 Collaborative filtering and matrix factorisation

[**TODO:** Neighbourhood methods versus latent-factor models. Matrix factorisation as the dominant latent-factor approach: given a partially observed user–item matrix $\mathbf{U} \in \mathbb{R}^{m \times n}$, find $\mathbf{A} \in \mathbb{R}^{m \times d}$ and $\mathbf{B} \in \mathbb{R}^{d \times n}$ with $\mathbf{U} \approx \mathbf{A} \cdot \mathbf{B}$. Cite koren2009 for the technique, hu2008 for implicit feedback. Note liang2018 (neural CF) and dacrema2019 (the reproducibility critique showing well-tuned MF remains competitive) — the latter matters because it justifies why the cryptographic literature targets MF rather than deep models.]

2.2 The recommendation pipeline

[**TODO:** THE ORGANISING DEVICE OF THIS SURVEY. Four stages: (1) COLLECTION – ratings leave the user; (2) TRAINING – a model is fitted to the collected data; (3) SERVING – scores are computed for a user and a ranking produced; (4) DELIVERY – the user actually fetches the recommended item. Emphasise that these are separable, that different papers protect different subsets, and that a system is only as private as its weakest stage.]

2.3 What the adversary sees

[**TODO:** Stage by stage, and this is where the motivation is earned rather than asserted: COLLECTION – raw ratings. TRAINING – the data set, and intermediate model state. SERVING – which user asked, and what was scored highly. DELIVERY – which item was fetched. Then the empirical evidence that this is not hypothetical: narayanan2008 de-anonymised the Netflix Prize data set against IMDb; calandrino2011 showed that the PUBLICLY VISIBLE OUTPUTS of a recommender — the “customers who bought this also bought” lists — leak individual transactions. calandrino2011 is the strongest single motivation in this survey: it attacks the system’s intended output, not a leak or a breach.]

2.4 Utility, and why it constrains the cryptography

[**TODO:** Brief. Recommendation quality is measured by ranking metrics ($\text{nDCG}@k$, $\text{Recall}@k$); a private recommender that recommends badly has solved nothing. This constrains protocol design — e.g. fixed-point arithmetic must not degrade the ranking — and it is why section 8 insists that quality be reported alongside cost.]

3 A Taxonomy of Approaches

[**TODO:** Open by stating the problem this section solves: papers in this field are routinely compared as though they were alternatives when they in fact protect different stages under different trust assumptions with incomparable guarantees. The four axes below are what make the comparison honest.]

[**TODO:** THE MASTER CLASSIFICATION TABLE — the single most valuable object in this survey. Rows: every surveyed system. Columns: trust model (axis 1) | privacy notion (axis 2) | pipeline stage(s) covered (axis 3) | adversary (axis 4) | evaluated scale. Tier C papers may appear here, since placement follows from title and abstract. Sort by pipeline stage so the sparsity of the delivery column is visible at a glance — that sparsity is a finding, and it should be readable from the table without prose. Use booktabs; this will need or a sideways float.]

Table 1: Classification of surveyed systems along the four axes of Section 3.

3.1 Axis 1: the trust model

[**TODO:** Who must be trusted not to collude? - SINGLE SERVER (no non-collusion assumption; needs HE or single-server PIR — computationally expensive) - TWO OR MORE NON-COLLUDING SERVERS (cheap information-theoretic or FSS-based protocols, at the price of an assumption about the world) - USER-SIDE / FEDERATED (no server holds raw data, but intermediate updates leak) - TRUSTED HARDWARE (assumes the enclave; side channels out of scope here) State plainly that these are not orderable by strength — they are different assumptions, not more or less of one.]

3.2 Axis 2: the privacy notion

[**TODO:** What does “private” formally mean in each case? - CRYPTOGRAPHIC (simulation-based; the real/ideal paradigm — cite lindell2017, evans2018). Hides everything but the agreed output. - DIFFERENTIAL PRIVACY (cite dwork2014; mcsherry2009 applied it to recommendation). Bounds an individual’s influence on the output; says nothing about who sees the computation. - ANONYMITY / MIXING (cite ishai2006 for the formal connection between anonymity and cryptographic strength). These compose but do not substitute. A system can be cryptographically private and still leak through its output — which is exactly calandrino2011’s attack, and the point section 9 returns to.]

3.3 Axis 3: which pipeline stage is protected

[**TODO:** Map onto Figure 1: collection / training / serving / delivery. THIS IS THE AXIS THE FIELD IS LEAST EXPLICIT ABOUT, and the one that makes the survey’s contribution. Most work protects training; considerably less protects delivery; very little protects both.]

3.4 Axis 4: the adversary

[**TODO:** Semi-honest versus malicious; static versus adaptive corruption; the collusion threshold. Note that almost all deployed-scale systems in this survey are semi-honest, and that this is a substantive limitation rather than a technicality.]

3.5 Reading the table

[**TODO:** Two or three paragraphs drawing out what the table shows — and ONLY what it shows. Expected observations, to be confirmed once the table is populated rather than assumed now: (i) the training column is dense and the delivery column sparse; (ii) the cheapest protocols consistently buy their speed with a non-collusion assumption; (iii) systems at the largest evaluated scale are semi-honest. [**VERIFY:** Do not write these until the table is filled — they are predictions, and the table may not support them.]]

[**TODO:** Figure: the GGM tree, showing the two parties' paths diverging only on the path to α . TikZ. This is the one figure a viewer of the video will remember — make it good.]

Figure 2: The GGM-tree construction of a $(2, 2)$ -distributed point function.

4 Cryptographic Building Blocks

[**TODO:** Frame the section by its punchline, stated up front: one primitive (FSS) turns out to be load-bearing on both sides of the problem. Every subsection below is evidence for that claim.]

4.1 Secret sharing

[**TODO:** Additive sharing over a ring; then 2-out-of-3 REPLICATED sharing (Araki et al., CCS'16), which is what NUDGE uses: $x = x_0 + x_1 + x_2$ with P_i holding (x_i, x_{i+1}) . Introduce $\llbracket x \rrbracket$ notation (macro in preamble.tex). Then the “3PC with PRF” model — each PAIR of parties shares a PRF key, so correlated randomness costs no communication.]

Observation 4.1. [**TODO:** THE key fact of the whole survey, stated as a numbered observation so later sections can point at it. Under replicated sharing, a degree-two function costs one round and $3db$ bits, and consequently a shared-matrix $\times$ shared-vector product communicates in proportion to the largest intermediate VECTOR, not to the size of the input MATRIX. NUDGE Thm 4.2. [**VERIFY:** quote the exact bound]]

4.2 Function secret sharing and distributed point functions

[**TODO:** Boyle–Gilboa–Ishai (EUROCRYPT'15, CCS'16). Define a $(2, 2)$ -DPF: $\text{Gen}(\alpha, \beta) \rightarrow (k_0, k_1)$ with $\text{Eval}(k_0, x) - \text{Eval}(k_1, x) = \beta$ iff $x = \alpha$, else 0, and each key alone pseudorandom. The GGM-tree construction in a paragraph plus one figure. Key size $d(\lambda + 2) + b$ bits — for $n = 4096$ that is ≈ 260 bytes against 32 KB for a one-hot vector. THAT RATIO IS THE POINT; give it as a concrete number, not asymptotically. Cite the July 2026 DPF/FSS survey (arXiv 2607.27696) as the entry point, and mention the current frontier: shorter information-theoretic keys (2604.24385), multi-party DDH constructions (2603.17453), GPU FSS (FuseFSS 2606.09551).]

4.3 FSS as a comparison gate

[**TODO:** Distributed comparison functions and zero-tests evaluated on shared inputs in a SINGLE round (NUDGE Table 3). Grotto (CCS'23) for fast comparison over $\mathbb{Z}_{2^n}$. This is what §4 needs.]

4.4 Private information retrieval

[**TODO:** Multi-server (information-theoretic, needs non-collusion) versus single-server (computational, needs LWE and a lot of compute). Hafiz–Henry (PoPETs'19.4) is the scheme PIRSONA builds on, and its computationally 1-private variant is DPF-compressed — so this is the SAME primitive as Section 4.2, in a different role. Say so explicitly; it is the section's thesis. Then SimplePIR/DoublePIR (USENIX Sec'23) and Spiral (S&P'22) for the single-server line, and Corrigan-Gibbs–Kogan (EUROCRYPT'20) for the preprocessing paradigm that §5 depends on.]

4.5 Oblivious RAM

[**TODO:** Keep this SHORT — it matters only for the indexed approaches in §5. Path ORAM (CCS’13), then Floram (CCS’17), Duoram (USENIX Sec’23) and PRAC (PoPETs’24). One sentence on why an index and obliviousness are in tension: an index is useful precisely because traversal is data-dependent, and data-dependent traversal is what obliviousness forbids.]

4.6 Fixed-point arithmetic over rings

[**TODO:** Why this deserves its own subsection rather than a footnote: it is where the real engineering cost sits. Represent $v \in \mathbb{R}$ as $\lfloor v \cdot 2^t \rfloor \in \mathcal{R}$; addition is free, multiplication needs a TRUNCATION afterwards, and repeated multiplication needs NORMALIZATION to stop values growing. Those two operations are the entire interactive cost of §4. Forward-reference it.]

5 Private Training

[**TODO:** Frame: everyone in this line computes the same thing — a low-rank factorisation $\mathbf{U} \approx \mathbf{A} \cdot \mathbf{B}$ — and the entire history is about which ALGORITHM for computing it survives being run under cryptography. That is the section’s thesis; state it first.]

5.1 Matrix factorisation, in the clear

[**TODO:** Half a page, no more. $\mathbf{U} \in \mathbb{R}^{m \times n}$, embedding dimension $d \ll \min(m, n)$, minimise $\|\mathbf{U} - \mathbf{A} \cdot \mathbf{B}\|_F$. Note that when $\mathbf{B}$ is orthonormal, $\mathbf{A} = \mathbf{U} \cdot \mathbf{B}^\top$, so recommendations for user i depend only on $\mathbf{u}^{(i)}$ and $\mathbf{B}$. [**VERIFY:** NUDGE §2] That fact does a lot of work later — flag it here and return to it in §8.]

5.2 Garbled circuits: Nikolaenko et al.

[**TODO:** CCS’13. Gradient descent inside a garbled circuit. Establish it as the baseline the field has been escaping ever since — NUDGE reports beating this family by FOUR ORDERS OF MAGNITUDE in communication and compute. [**VERIFY:** exact claim]. Say why: data-oblivious gradient descent forces a circuit whose size scales with the whole rating matrix.]

5.3 PIRSONA: four-party Boolean factorisation

[**TODO:** PoPETs’21. PIRSONA [VBH21] — the instructor’s paper, so be thorough. Cover: the 4PC Boolean matrix factorisation; the new primitives it contributes (4PC fixed-selection MUX/DeMUX, fast 3PC vector normalisation for shared fixed-point vectors, 4PC well-formedness tests for (2, 2)-DPFs, one-round 3PC integer comparison); and the pairwise non-collusion assumption it pays for them. Its DELIVERY half belongs in §6, not here — cross-reference, do not duplicate.]

5.4 NUDGE: power iteration as a matrix-vector program

[**TODO:** USENIX Sec’26. The current state of the art, and the section’s centrepiece. Build the argument in this order: (1) recall Theorem 4.1 — matrix-vector work is free; (2) therefore prefer an algorithm that is mostly matrix-vector products separated by cheap non-linear steps; (3) power iteration has exactly that shape and

Algorithm 1 ApproxFactor: matrix factorisation by power iteration [HDCB26]

[**TODO:** Transcribe NUDGE Fig. 4 faithfully, with a citation. Mark clearly which lines are FREE (the two Mul calls) and which are INTERACTIVE (Normalize). Annotate the line where each row of **B** is OPENED IN THE CLEAR — that is the design’s load-bearing trick and the hinge of §8.]

[**TODO:** Table: system $\times$ {parties, corruption model, algorithm, dataset, scale reached, cost, quality metric}. Include Nikolaenko, PIRSONA, NUDGE, federated, FHE. Numbers must carry their SETTING — hardware and network — or they are not numbers.]

Table 2: Private training systems compared.

gradient descent does not. THIS IS THE POINT OF THE WHOLE SECTION. Then the matrix-vector program abstraction (Def 4.1) and Thm 4.2.]

5.4.1 The two protocols that cost anything

[**TODO:** Trunc_t : deterministic truncation with sign extension; low-order carries fixed by an integer comparison, high-order carries prevented by one bit of slack. 3 rounds, $2dt(\lambda + 4) + 10db$ bits. Emphasise WHY that is an improvement: the leading term is λt rather than λb , giving $\approx b/t = 6\times$ at $b = \lambda = 128$, $t = 20$. ApproxNormalize: shares of $1/\|v\|$. $\|v\|^2$ is degree-two hence free; the inverse square root is seeded from the most-significant-non-zero-bit via $b + 1$ SIMULTANEOUS comparisons in one round, then Newton–Raphson. Explain what this avoids — the $O(b)$ -round route and the exponential-lookup-table route. [**VERIFY:** all constants against NUDGE §4.3, Lemmas 4.3 and 4.4]]

5.5 The non-cryptographic alternatives

[**TODO:** Short — one paragraph each, for contrast and completeness. Federated MF (Chai et al.) and its leakage through gradients; FHE-based MF (arXiv 2509.03024) and its cost; Paillier/decentralised variants (PADER, 2601.10212); recent secret-sharing recommenders (PRSI, 2411.18653). The purpose here is to show we surveyed the field, and to justify why §8 targets the MPC line.]

5.6 Where training now stands

[**TODO:** Close with: private training is essentially SOLVED at real scale — Netflix, 0.5M users, 50 minutes, quality on par with non-private MF. Then the pivot that sets up §5, quoting NUDGE §3.1 verbatim: it “relies on other means... to let users fetch data items in a private way.” End the section on that quote.]

6 Private Retrieval

[**TODO:** Open by connecting to §4’s closing quote: here is what “other means” actually contains. Then the framing — serving a recommendation is a nearest-neighbour query, so this literature and the recommendation literature are solving adjacent problems without citing each other.]

6.1 The problem: private nearest-neighbour search

[**TODO:** Table: system $\times$ {setting (1-server/2-server/ORAM), online cost, client storage, corpus scale, privacy notion}. The privacy-notion column is what makes this table worth printing — these systems are NOT directly comparable and the table should show why.]

Table 3: Private retrieval systems compared.

[**TODO:** Formalise. A client holds query vector q ; a server holds a corpus of vectors; the client wants the top- k by inner product or distance, and the server must learn nothing about q or the result. Then the DEFINING difficulty, which is the same lesson as Section 4.5: in the clear you use an index (k-d tree, LSH, IVF, HNSW), and an index is useful precisely BECAUSE traversal is data-dependent. Obliviousness forbids exactly that. So you either scan linearly and lose the index, or keep the index and need ORAM. State this as the organising axis of the whole section.]

6.2 Linear-scan approaches

[**TODO:** SANNS (USENIX Sec'20) as the origin point — secure approximate k -NN combining HE with garbled circuits. Then Tiptoe (SOSP'23): private search over a web-scale corpus, built on SimplePIR-family machinery. Note the same first author as NUDGE. The barrier: the server must touch EVERY entry per query, so throughput is bounded — [**VERIFY:** Wally reports Tiptoe at 909 QPS with 17.4 MB communication on a 3.2M-entry database; check against the Wally paper before citing]. Everything in §6.3 exists to break that bound.]

6.3 Sublinear and indexed approaches

[**TODO:** Pacmann (eprint 2024/1600): graph-based ANN where the client walks the graph, privately retrieving local neighbourhood information at each hop using preprocessing-PIR (Section 4.4). Sublinear online, but [**VERIFY:** 614 MB client storage and an offline phase streaming the whole database] — state the cost honestly, it is the tradeoff that matters. Compass (NSDI'24): ORAM over HNSW. Strongest privacy in the group, heaviest cost. MESS (arXiv 2607.28999, July 2026): the newest in this line, multi-graph HNSW. Being current here is cheap and visible.]

6.4 Relaxing the guarantee

[**TODO:** Wally (arXiv 2406.06761): uses DIFFERENTIAL PRIVACY plus batching to break the linear-computation barrier, at the price of a weaker, DP-style guarantee rather than full obliviousness. Panther (CCS'25): private ANN in the single-server setting. Make the tradeoff explicit — this subsection is where the survey shows it understands that “private” is a dial, not a switch. Refer back to ??.]

6.5 Private top- k as a service

[**TODO:** The retrieval-augmented-generation line, which arrived at the same problem from a different direction and is worth including because it shows the primitive is in demand beyond recommendation: PIR-RAG (arXiv 2509.21325) and P²RAG (arXiv 2603.14778, arbitrary top- k). Also VIPIR (2606.11536) for GPU-accelerated PIR throughput.]

6.6 An observation about the recommendation setting

[**GAP-ARGUMENT:** First half of the §8 argument. Set it up here; do not resolve it.]

[**TODO:** Figure: the PIRSONA cycle — fetch → harvest → factorise → recommend → fetch. One loop, drawn once. TikZ. This figure and Figure 2 are the two the video needs.]

Figure 3: The PIRSONA delivery–training cycle.

[**TODO:** Every system above assumes the CORPUS is the server’s secret and the QUERY is the client’s. But in the NUDGE setting the item embedding matrix $\mathbf{B}$ is PUBLIC by construction (Algorithm 1), and the user holds its own ratings $\mathbf{u}^{(i)}$. So the recommendation-serving instance of this problem is strictly easier than the general one — and in some parameter regimes it may not need cryptography at all. State the question and stop: how large must the catalogue be before that stops being true? Answer it in ??.]

7 Systems That Attempt Both Halves

[**TODO:** Open by restating Section 1.1: a recommender that protects its users must do BOTH. §4 surveyed one half, §5 the other. This section asks who has done both — and the answer is very nearly nobody.]

7.1 PIRSONA as an end-to-end system

[**TODO:** Now cover PIRSONA’s DELIVERY half, deferred from Section 5.3. Three things: (1) Hafiz–Henry multiserver PIR for the fetch, in its computationally 1-private DPF-compressed variant; (2) THE HARVEST LOOP — the servers extract secret-shared consumption histories DIRECTLY from the incoming PIR queries, so training data arrives as a by-product of delivery with no separate upload. No other system in either §4 or §5 does this. Give it its own paragraph; it is the most reusable idea in the paper. (3) users hold no local state and never actively participate in training — contrast with federated learning explicitly.]

7.2 Partial attempts

[**TODO:** Honest accounting of near-misses, so the gap claim is not overstated: PrivateRec (arXiv 2204.08146) names both training and serving, but is differentially private and federated rather than MPC + PIR — a different point in the ?? taxonomy. Trusted-hardware deployments do both, by assuming the problem away. Anonymity networks (Tor, private relay) are what NUDGE actually suggests, and provide traffic-analysis-vulnerable anonymity rather than cryptographic privacy. Say why that is a weaker guarantee.]

7.3 The intersection is nearly empty

[**VERIFY:** Before this claim ships, run a forward-citation sweep of NUDGE, Tiptoe and Pacmann. If someone has already composed a modern private-ANN backend with a private-MF recommender, we must find it NOW, not in October. This is the single most falsifiable claim in the survey.]

[**TODO:** State the finding plainly and without overclaiming: (1) PIRSONA (2021) is essentially the only system spanning both halves cryptographically; (2) its training core has since been superseded — NUDGE reports beating the bespoke four-party protocol with 8× less communication and ONE FEWER non-colluding party; (3) its retrieval core predates the entire §5 literature, all of which is 2023 or later; (4) NUDGE improves half the problem and explicitly declines the other half. Therefore nobody has built a system with a modern training core AND a modern retrieval layer. That sentence is the survey’s finding.]

[**TODO:** Table: the two-by-two. Rows = private training yes/no, columns = private delivery yes/no, cells = the systems. The point of the table is the sparse cell. Make it visually obvious — this is the slide that ends the video.]

Table 4: Coverage of the two halves across the surveyed systems.

8 How This Literature Is Evaluated

[**TODO:** Open by justifying the section's existence: the numbers in §4 and §5 are not comparable unless the reader knows what was held fixed. This section makes the comparison rules explicit — and commits us to them.]

8.1 Metrics that matter

[**TODO:** Four families, each with an example from the surveyed work: (a) wall-clock — but always paired with hardware. NUDGE's 50 minutes is on THREE 192-CORE MACHINES; the number is meaningless without that. (b) communication — server-to-server bytes (40 GB for NUDGE on Netflix) and per-client bytes (298 KB serving). (c) client-side cost — storage and bandwidth. This is what separates Pacmann (614 MB client storage) from Tiptoe. (d) QUALITY — $nDCG@k$, $Recall@k$. A private recommender that recommends badly has solved nothing. NUDGE scores 0.29 $nDCG@20$ against 0.31 for non-private neural models; that gap IS the result.]

8.2 Network models

[**TODO:** Make the point sharply, because it is where course projects fail: results in this literature FLIP under network constraints, because round complexity starts to dominate bandwidth. A localhost number tells you almost nothing about a deployment. Standard practice is `tc netem` with LAN and WAN profiles — e.g. 30 ms/100 Mbps and 100 ms/10 Mbps. Note which surveyed papers report WAN numbers and which report LAN only. NUDGE's headline is LAN. [**VERIFY:** check each paper's setup section before asserting this]]

8.3 Baselines

[**TODO:** The three baselines this literature expects, and why each is needed: (a) the plaintext system — gives the honest slowdown factor; (b) a generic-MPC-framework implementation (MP-SPDZ) — the “why not just use a framework” answer; (c) at least one published secure system — the credibility test. Observe that the strongest papers report the UNFLATTERING comparison too. A slowdown stated plainly reads as confidence; a slowdown omitted reads as evasion.]

8.4 The standard we hold ourselves to

[**TODO:** Write this as a commitment, in the first person, and keep it to five bullets. It is quoted back at us in the final report: (1) a written threat model and an explicit leakage profile; (2) baselines including the unflattering ones; (3) WAN numbers, not just localhost; (4) a microbenchmark breakdown — where do the milliseconds go; (5) reproducibility: one command per figure, a README a stranger can follow. None of these requires cryptographic novelty, and most groups will do none of them.]

9 Open Problems

[**TODO:** One framing paragraph: these are drawn from limitations the surveyed authors state themselves, and from gaps visible in Table 1. Say so — it is what separates a survey’s open-problems section from speculation.]

9.1 Malicious security at scale

[**TODO:** Nearly every system evaluated at realistic scale is semi-honest. nudge2026 lists malicious security as its first open question; grotto2023 notes that pika2022 generalised its predecessor technique to the malicious setting at a cost. Report what the papers say the barrier is, without asserting a solution.]

9.2 Reducing the non-collusion assumption

[**TODO:** The recurring trade: the fastest protocols assume two or three non-colluding servers. nudge2026 explicitly poses serving private recommendations with only two non-colluding parties as open. Contrast with the single-server line (simplepir2023, spiral2022), which removes the assumption at a computational cost. Present as a trade-off the field has not resolved, not as a defect.]

9.3 The delivery stage

[**TODO:** The sparse column in Table 1. nudge2026 states in its non-goals that it relies on “other means” — private relays, Tor, or cryptographic PIR — for users to fetch items privately. pirsona2021 is one of the few systems to address collection, training and delivery together. Meanwhile a substantial private-retrieval literature has developed largely separately (section 6). Report this as an observation about the literature; it is the gap our own project engages with, but this section is not the place to argue for that.]

9.4 Leakage through the recommendation itself

[**TODO:** The deepest open problem, and worth stating carefully because it is not a protocol failure. calandrino2011 attacks the recommender’s INTENDED OUTPUT: no protocol flaw is involved. A system can be cryptographically perfect and still leak, because the functionality itself is revealing. asharov2018 make a structurally similar point for genomic similarity search. Cryptography bounds who sees the computation; it does not bound what the output implies. DP addresses this (mcsherry2009, and the DP extension in nudge2026) at a utility cost. Note that composing cryptographic and differential privacy in recommendation remains only partly explored.]

9.5 Evaluation and comparability

[**TODO:** Forward from section 8: the field lacks a shared benchmark. Papers use different data sets, hardware, network models and quality metrics, so published numbers are frequently not comparable. This is a tractable, unglamorous open problem and worth naming as one.]

9.6 Scope of our own follow-on work

[**TODO:** THREE OR FOUR SENTENCES, NO MORE. The course project that follows this survey engages with Section 9.3. Do not pitch, do not claim novelty, do not promise results — state the interest and stop. Overselling here would undercut the survey’s credibility.]

10 Conclusion

[**TODO:** Three short paragraphs and stop. (1) What the survey found: private training is essentially solved at real scale, private retrieval has moved enormously since 2021, and the two have not been connected. (2) The two questions that fall out of that observation (??, ??), and why both are answerable within this project rather than aspirational. (3) What we will do next, in two sentences. Resist a grand closing claim. The survey's value is the map and the gap; overselling it here undoes the credibility ?? just bought.]

References

- [HDCB26] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Dan Boneh. Nudge: A private recommendations engine. In *35th USENIX Security Symposium, USENIX Security 2026, Baltimore, MD, USA, August 12-14, 2026*. USENIX Association, 2026. Full version: IACR ePrint 2026/179.
- [VBH21] Adithya Vadapalli, Fattaneh Bayatbabolghani, and Ryan Henry. You may also like... privacy: Recommendation systems meet PIR. *Proc. Priv. Enhancing Technol.*, 2021(4):30–53, 2021.